Développement d'un système embarqué pour mesurer et estimer la VO2 max à partir des tests de Léger et Cooper

Annexe : Code Heltec recepteur Cooper

STN « 2025» _ « 2026 »

Projet : STN21 – Système embarqué VO_2max

Auteur: Ben said Wassim

```
#include <RadioLib.h>            // Include RadioLib library for LoRa radio
communication
#include <Wire.h>                // Include Wire library for I2C communication
#include "SSD1306Wire.h"         // Include SSD1306 OLED display library
#include "pins_arduino.h"        // Include Arduino pin definitions

// ==================== LoRa ====================
#define LORA_FREQ  915.0             // LoRa frequency in MHz (915 MHz for US
region)
#define LORA_SS    8                  // Chip Select (CS) pin for LoRa module
#define LORA_RST   12                 // Reset pin for LoRa module
#define LORA_DI0   14                 // Interrupt pin DIO0 for LoRa module

// Create LoRa module with defined pins
Module loraModule(LORA_SS, LORA_DI0, LORA_RST);
// Create radio object for SX1262 chip
SX1262 radio(&loraModule);

// ==================== OLED ====================
// Create OLED display object with I2C address 0x3C, using SDA and SCL pins
SSD1306Wire display(0x3C, SDA_OLED, SCL_OLED);

/**
 * Enables power supply for the OLED display
 * Vext is the external power control pin
 * Logic is inverted: LOW = enabled, HIGH = disabled
 */
void VextON() {
  pinMode(Vext, OUTPUT);      // Configure pin as output
  digitalWrite(Vext, LOW);    // Set to LOW to enable power
}

/**
 * Resets the OLED display via its reset pin
 * Ensures the display starts up correctly
 */
void displayReset() {
  pinMode(RST_OLED, OUTPUT);   // Configure reset pin as output
  digitalWrite(RST_OLED, HIGH); // Set high initially
  delay(1);
  digitalWrite(RST_OLED, LOW);  // Pulse low for reset
  delay(1);
  digitalWrite(RST_OLED, HIGH); // Return to high state
  delay(1);
}

// ==================== Data Variables ====================
// Variables to store the latest received data
float lastLat=0, lastLon=0, lastAlt=0;      // Last received GPS coordinates
float lastDist=0, lastVO2=0;                  // Last distance and VO2max
values
int lastSat=0;                                  // Last number of satellites
```

```
String lastTime="N/A";                          // Last received timestamp

// 12-minute chronometer management
unsigned long startTime = 0;                    // Timestamp when reception
started
bool chronoStarted = false;                      // Indicates if chronometer
has started
const unsigned long DUREE_TEST = 12UL * 60UL * 1000UL; // 12 minutes in
milliseconds

// ==================== Setup ====================
void setup() {
  // Initialize serial communication for debugging
  Serial.begin(115200);
  delay(500);                                    // Small delay for
stabilization
  Serial.println("=== Récepteur LoRa + OLED ==="); // Print startup message

  // ---- OLED Initialization ----
  VextON();                                     // Enable OLED power supply
  displayReset();                                // Reset OLED display
hardware
  display.init();                                // Initialize OLED display
  display.flipScreenVertically();                // Set vertical orientation
(based on mounting)
  display.setFont(ArialMT_Plain_10);              // Set standard font size
  display.clear();                               // Clear display buffer
  display.drawString(0,0,"Init OLED + LoRa...");  // Display initial message
  display.display();                             // Update physical display

  // ---- Manual LoRa Module Reset ----
  pinMode(LORA_RST, OUTPUT);                     // Configure reset pin as
output
  digitalWrite(LORA_RST, LOW);     // Set LOW to trigger reset
  delay(10);
  digitalWrite(LORA_RST, HIGH);    // Return to HIGH to start module
  delay(10);

  // Initialize LoRa radio
  int state = radio.begin(LORA_FREQ);            // Start radio at defined
frequency
  if(state != 0) {                  // If initialization failed
    display.drawString(0,20,"Erreur LoRa !");    // Display error message
    display.display();
    while(1);                       // Halt program execution
  }

  // Confirm successful initialization
  display.drawString(0,20,"LoRa OK !");          // Display success message
  display.drawString(0,32,"En attente...");      // Display waiting message
  display.display();

  // Start the 12-minute chronometer
  startTime = millis();             // Record start time
  chronoStarted = true;              // Activate chronometer
}
```

```
// ==================== Loop ====================
void loop() {
  String packet;                        // Variable to store received packet
  // Attempt to receive a LoRa packet
  int state = radio.receive(packet);

  // If reception successful (no error)
  if(state == RADIOLIB_ERR_NONE) {
    // Display received packet in serial monitor
    Serial.println("Paquet recu: " + packet);

    // Find indexes of different fields in the packet
    int iLat  = packet.indexOf("LAT:");      // Index of Latitude field
    int iLon  = packet.indexOf(",LON:");     // Index of Longitude field
    int iAlt  = packet.indexOf(",ALT:");     // Index of Altitude field
    int iSat  = packet.indexOf(",SAT:");     // Index of Satellites field
    int iDist = packet.indexOf(",DIST:");    // Index of Distance field
    int iVO2  = packet.indexOf(",VO2:");     // Index of VO2max field
    int iTime = packet.indexOf(",TIME:");    // Index of Time field

    // Verify that all fields were found
    if(iLat!=-1 && iLon!=-1 && iAlt!=-1 && iSat!=-1 &&
       iDist!=-1 && iVO2!=-1 && iTime!=-1) {

      // Extract and convert different values
      // Each field is extracted between its marker and the next one
      lastLat  = packet.substring(iLat+4, iLon).toFloat();     // Latitude
(LAT:xxx,)
      lastLon  = packet.substring(iLon+5, iAlt).toFloat();     // Longitude
(LON:xxx,)
      lastAlt  = packet.substring(iAlt+5, iSat).toFloat();     // Altitude
(ALT:xxx,)
      lastSat  = packet.substring(iSat+5, iDist).toInt();      // Satellites
(SAT:xxx,)
      lastDist = packet.substring(iDist+6, iVO2).toFloat();    // Distance
(DIST:xxx,)
      lastVO2  = packet.substring(iVO2+5, iTime).toFloat();    // VO2max
(VO2:xxx,)
      lastTime = packet.substring(iTime+6);                    // Time
(TIME:xxx) until end

      // ---- Update OLED Display ----
      display.clear();                                          // Clear
display buffer
      display.setFont(ArialMT_Plain_10);                        // Set
standard font
      display.drawString(0,0,"GPS RX LoRa OK");                 // Confirm
reception
      display.drawString(0,12,"UTC: " + lastTime);              // Display UTC
time
      display.drawString(0,24,"Sat: " + String(lastSat));       // Display
number of satellites
      display.drawString(0,36,"Dist: " + String(lastDist,1) + " m"); //
Display distance (1 decimal)
```

```
      display.drawString(0,48,"VO2: " + String(lastVO2,1));     // Display
VO2max (1 decimal)
      display.display();                                         // Update
physical display
    }
  }

  // ==== Check for 12-minute Completion ====
  // If chronometer has started AND elapsed time exceeds 12 minutes
  if(chronoStarted && (millis() - startTime >= DUREE_TEST)) {
    // Display final summary on OLED
    display.clear();
    display.setFont(ArialMT_Plain_10);
    display.drawString(0,10,"=== FIN TEST ===");           // Display test end
message
    display.drawString(0,28,"Distance: " + String(lastDist,1)); // Display
final distance
    display.drawString(0,40,"VO2: " + String(lastVO2,1));       // Display
final VO2max
    display.display();

    // Display final summary on serial monitor
    Serial.println("===== 12 minutes écoulées =====");     // Print test end
message
    Serial.println("Distance totale : " + String(lastDist)); // Print total
distance
    Serial.println("VO2 max : " + String(lastVO2));          // Print final
VO2max

    // Halt program execution
    while(1);   // Infinite loop to stop execution
  }
}
```